

Objectives

- Explain the need and Benefits of component life cycle
- Identify various life cycle hook methods
- List the sequence of steps in rendering a component

In this hands-on lab, you will learn how to:

- Implement componentDidMount() hook
- Implementing componentDidCatch() life cycle hook.

Prerequisites

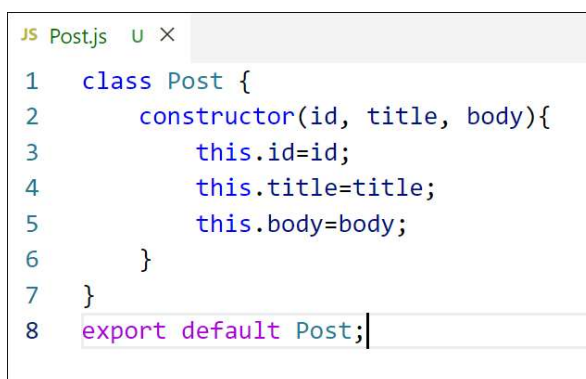
The following is required to complete this hands-on lab:

- Node.js
- NPM
- Visual Studio Code

Notes

Estimated time to complete this lab: **60 minutes**.

1. Create a new react application using *create-react-app* tool with the name as “blogapp”
2. Open the application using VS Code
3. Create a new file named as **Post.js** in **src** folder with following properties

A screenshot of a code editor window titled 'JS Post.js'. The code defines a class 'Post' with a constructor that takes 'id', 'title', and 'body' as arguments and assigns them to 'this.id', 'this.title', and 'this.body' respectively. The class is then exported as the default export.

```
1 class Post {
2   constructor(id, title, body){
3     this.id=id;
4     this.title=title;
5     this.body=body;
6   }
7 }
8 export default Post;
```

Figure 1: Post class

4. Create a new class based component named as **Posts** inside **Posts.js** file

```

JS Posts.js U X
1 class Posts extends React.Component {
2   constructor(props){
3     super(props);
4   }
5 }

```

Figure 2: Posts Component

5. Initialize the component with a list of Post in state of the component using the constructor
6. Create a new method in component with the name as **loadPosts()** which will be responsible for using Fetch API and assign it to the component state created earlier. To get the posts use the url (<https://jsonplaceholder.typicode.com/posts>)

```

JS Posts.js U X
1 class Posts extends React.Component {
2   constructor(props){
3     super(props);
4     //code
5   }
6   loadPosts() {
7     //code
8   }
9 }

```

Figure 3: loadPosts() method

7. Implement the **componentDidMount()** hook to make calls to **loadPosts()** which will fetch the posts

```

JS Posts.js U X
1 class Posts extends React.Component {
2   constructor(props){
3     super(props);
4     //code
5   }
6   loadPosts() {
7     //code
8   }
9   componentDidMount() {
10    //code
11  }
12 }

```

Figure 4: componentDidMount() hook

8. Implement the **render()** which will display the title and post of posts in html page using heading and paragraphs respectively.

```
JS Posts.js U X
1  class Posts extends React.Component {
2  >    constructor(props) { ...
5      }
6  >    loadPosts() { ...
8      }
9  >    componentDidMount() { ...
11     }
12     render() {
13         //code
14     }
15 }
```

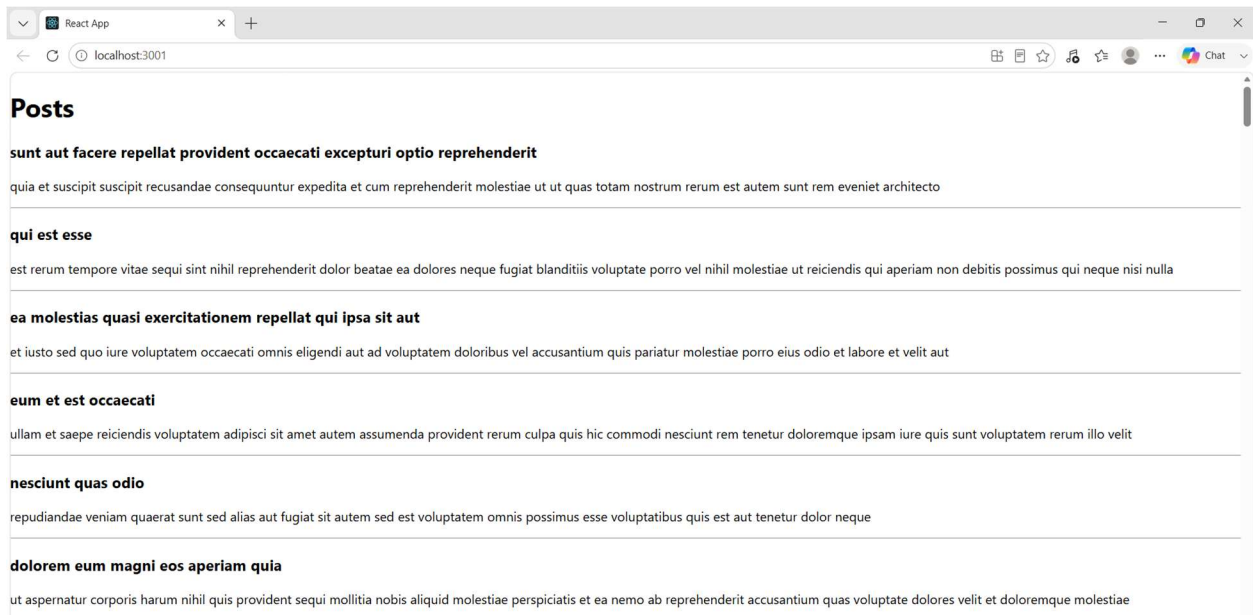
Figure 5: *render()* method

9. Define a **componentDidCatch()** method which will be responsible for displaying any error happening in the component as alert messages.

```
JS Posts.js U X
1  class Posts extends React.Component {
2  >    constructor(props) { ...
5      }
6  >    loadPosts() { ...
8      }
9  >    componentDidMount() { ...
11     }
12 >    render() { ...
14     }
15     componentDidCatch(error, info) {
16         //code
17     }
18 }
```

Figure 6: *componentDidCatch()* hook

10. Add the Posts component to App component.
11. Build and Run the application using *npm start* command.



Step 1

Create a React project.

```
npx create-react-app blogapp
```

Step 2

Move into the project folder.

```
cd blogapp
```

Step 3

Open the project in Visual Studio Code.

```
code .
```

Step 4

Create **Post.js** inside the **src** folder and define the Post class.

Step 5

Create **Posts.js** inside the **src** folder.

Step 6

Initialize the component state with an empty array.

Step 7

Create the **loadPosts()** method to fetch data from:

<https://jsonplaceholder.typicode.com/posts>

Step 8

Use the **componentDidMount()** lifecycle method to call `loadPosts()`.

Step 9

Implement the **render()** method to display all posts.

Step 10

Implement **componentDidCatch()** to handle runtime errors.

Step 11

Import the **Posts** component into **App.js**.

Step 12

Run the application.

`npm start`

Project Structure

blogapp

|

|— node_modules

|— public

|— src

| |— App.js

| |— Post.js

| |— Posts.js

| |— index.js

|— package.json

|— package-lock.json

```
JS post.js  JS posts.js  X  JS App.js

src > JS posts.js > ...
1  import React, { Component } from 'react';
2  import Post from './p|pst';
3
4  class Posts extends Component {
5
6      constructor(props) {
7          super(props);
8
9          this.state = {
10             posts: []
11         };
12     }
13
14     loadPosts() {
15         fetch("https://jsonplaceholder.typicode.com/posts")
16             .then(response => response.json())
17             .then(data => {
18                 const posts = data.map(item =>
19                     new Post(item.id, item.title, item.body)
20                 );
21
22                 this.setState({
23                     posts: posts
24                 });
25             })
26             .catch(error => {
27                 alert(error);
28             });
29     }
30
31     componentDidMount() {
32         this.loadPosts();
33     }
34 }

JS post.js  X  JS posts.js  JS App.js

src > JS post.js > [?] default
1  class Post {
2      constructor(id, title, body) {
3          this.id = id;
4          this.title = title;
5          this.body = body;
6      }
7  }
8
9  export default Post;
```

```
JS post.js  JS posts.js  JS App.js  X
src > JS App.js > App
1  import Posts from './posts';
2
3  function App() {
4    return (
5      <div>
6        <Posts />
7      </div>
8    );
9  }
10
11 export default App;
```